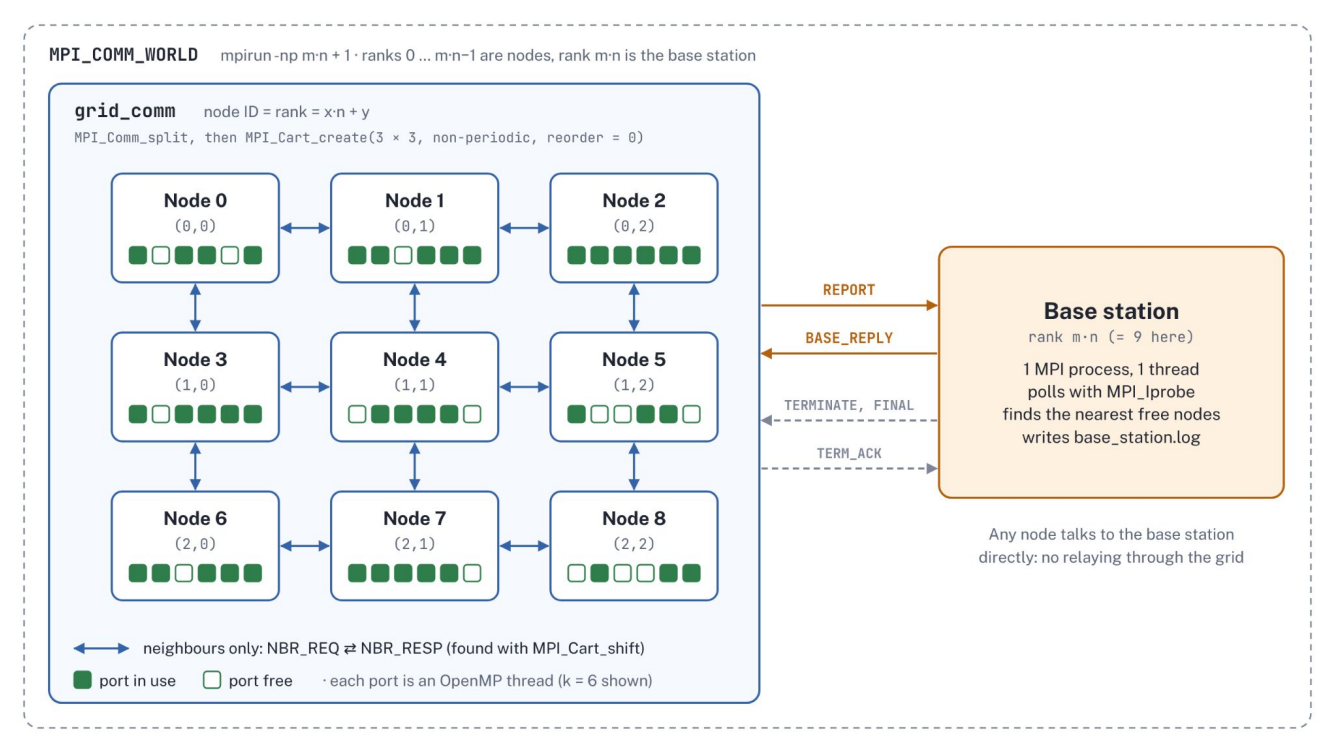
FIT3143 Lab 3

Julia Mallard, Nam Quang Tran

Task 1: EV Charging Nodes

System architecture



MPI_Comm_split splits the base station away from the nodes to put the nodes in the same communicator.

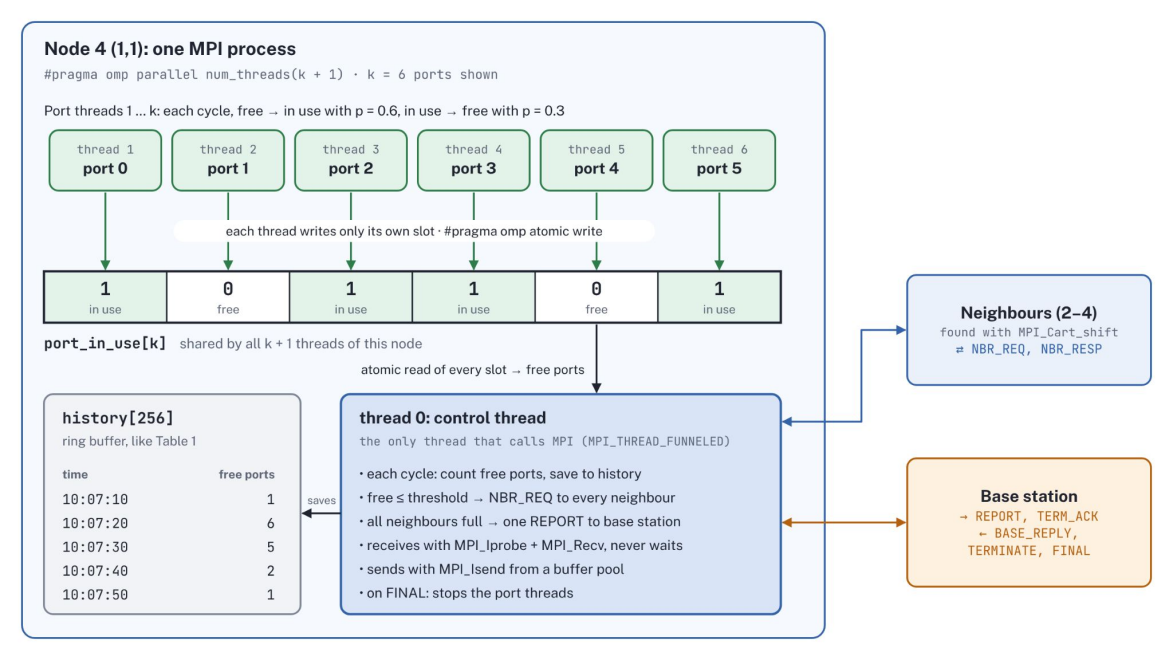
Use MPI_Cart_Create to form the cartesian topology.

MPI_Cart_shift to find neighbours (looking up/down, left/right).

Grid is not periodic, neighbours should be able to wrap around the edges, 6 is not a neighbour of 0 for example.

m·n + 1 processes · Cartesian topology · 2–4 neighbours · direct link to base station

Inside a charging node



Controller checks number of free ports each cycle (1 sec in the code).

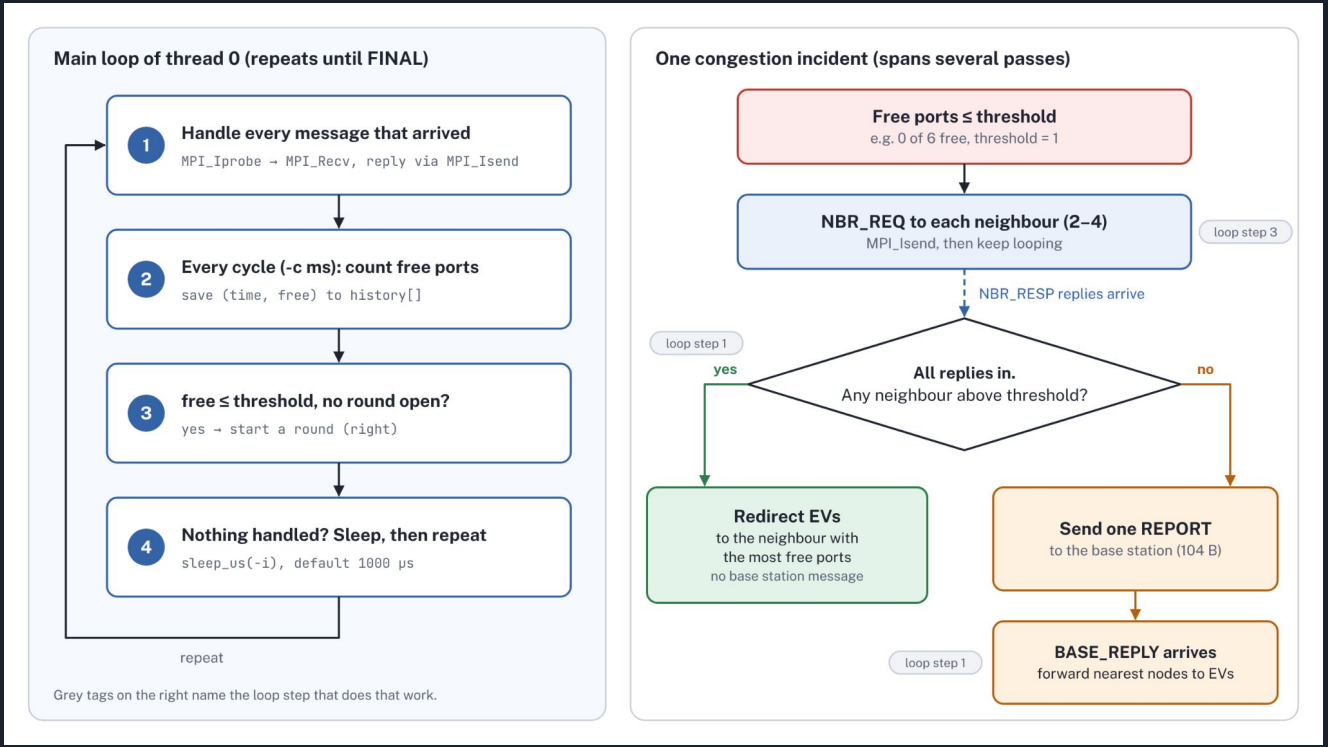
If free <= threshold, report to every neighbour with NBR_REQ, neighbours report back with NBR_RESP.

MPI_Isend to send messages (immediately, non blocking), MPI_Iprobe to check if messages have arrived, MPI_Recv to receive messages.

k + 1 OpenMP threads · shared port_in_use[], one writer per slot · thread 0 is the only MPI thread

Each cycle a port fills with probability 0.6 or frees with probability 0.3 per cycle.

Handling congestion



ask neighbours → redirect, or one REPORT · 66 of 72 incidents solved by neighbours

Message passing

Message	From -> To	Size
NBR_REQ / NBR_RESP	Node <-> neighbour	12 B/ 16 B
REPORT	Node -> Base	104 B
BASE_REPLY	Base -> Node	540 B
TERMINATE / TERM_ACK/FINAL	shutdown	0/156/0 B

Thread 0 loop

while not FINAL:

  while MPI_Iprobe(any source, any tag): // handle everything that arrived

    MPI_Recv, then react (request / reply / base reply / terminate)

  every cycle:

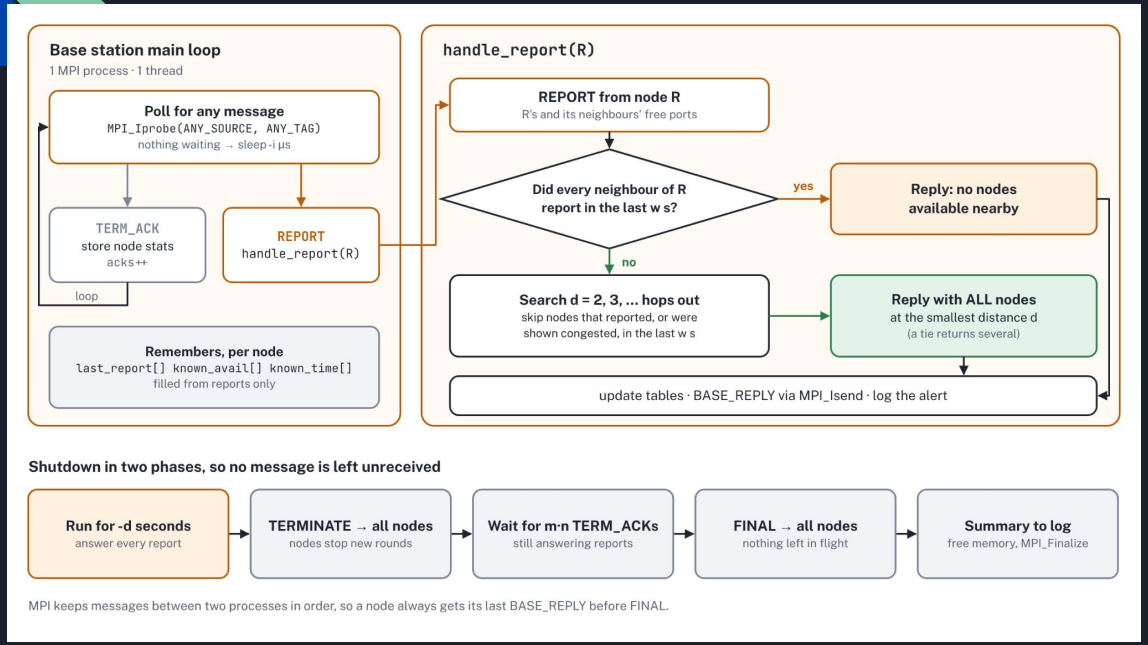
    free = free slots in port_in_use

    if free <= threshold: MPI_Isend NBR_REQ to each neighbour

  if nothing arrived: sleep one poll interval

Task 2: Base Station

Base station



window rule · search from 2 hops · return all nearest nodes · logging

Waits for REPORTs from congested nodes

Window rule: all of R's neighbours reported recently -> "none nearby"

Otherwise search 2, 3, … hops out, using memory only

Skip nodes that reported or were seen full recently

Reply with all nodes at the nearest distance

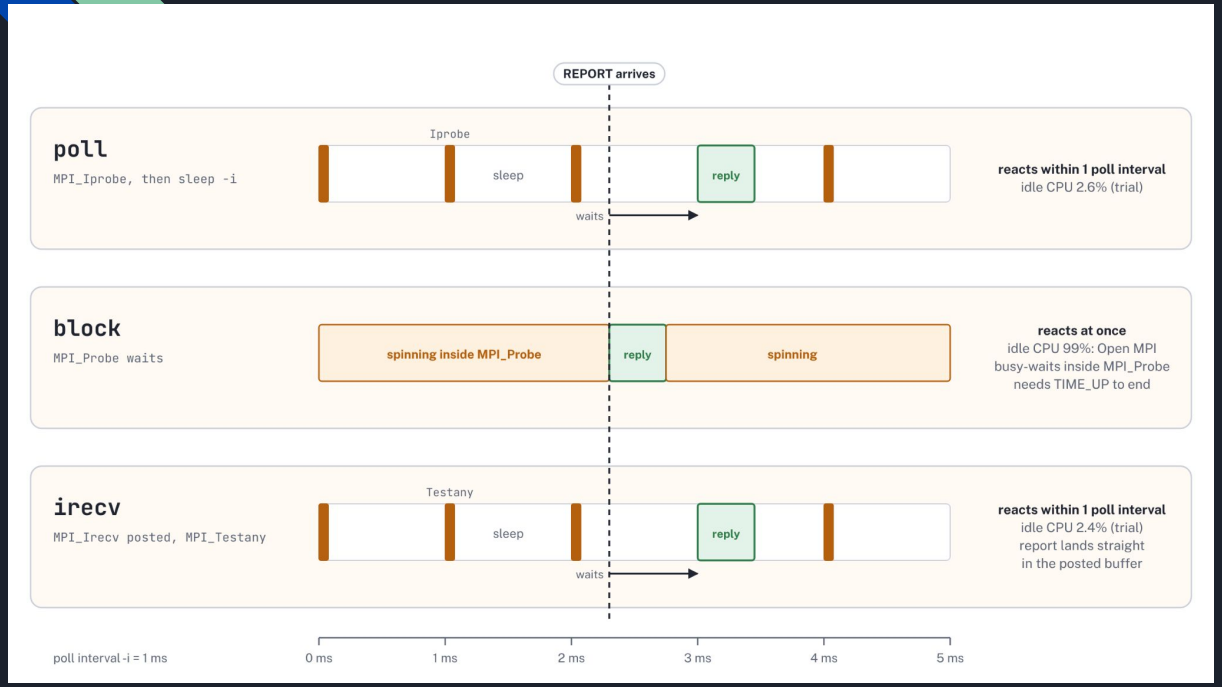
Log every alert, then a two-phase shutdown

Nearest-node search: real alerts



alert #5 → node 2 · alert #6 → none nearby

Base station versions (blocking vs non blocking)



poll: non-blocking, Iprobe + sleep (picked)

irecv: non-blocking, Irecv + Testany

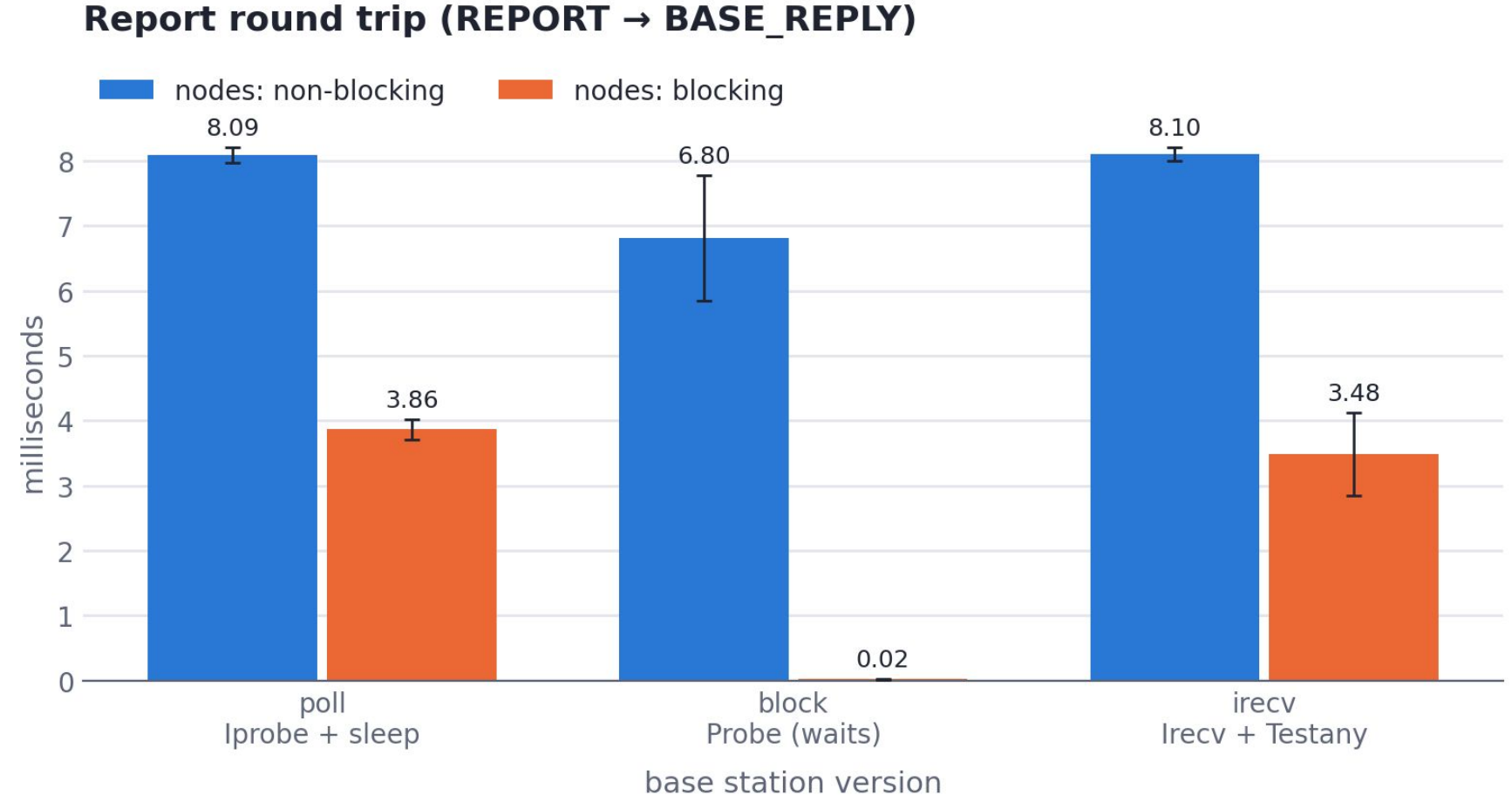
block: blocking, Probe

Nodes

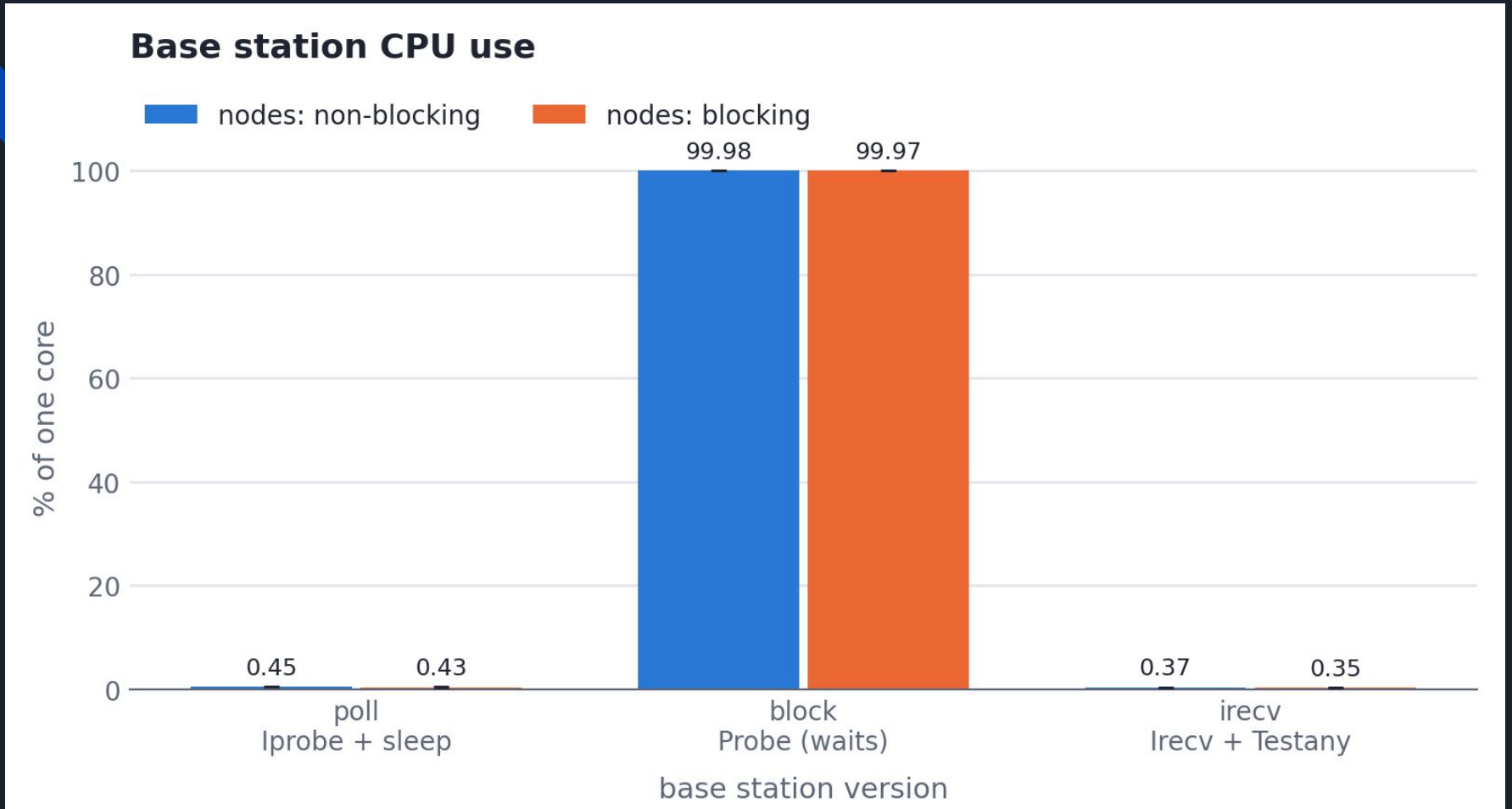
non-blocking: Isend + Iprobe (picked)

blocking: Send + Probe, still answers requests while waiting

Blocking base can’t watch the clock, so node 0 sends TIME_UP



4x4 grid, 15 s runs, cycle 500 ms, poll interval 1000 µs, 3 runs per bar; bars = mean, whiskers = ±1 standard deviation



4x4 grid, 15 s runs, cycle 500 ms, poll interval 1000 µs, 3 runs per bar; bars = mean, whiskers = ±1 standard deviation

Task 3: Communication Analysis

Benchmark Methodology & Setup

Single-Clock Round Trips:

- Measured on requesting node or base station clock (NBR_REQ → NBR_RESP, REPORT → BASE_REPLY net of Tproc, TERMINATE → TERM_ACK)
- Eliminates distributed clock synchronization errors

Benchmark Dataset:

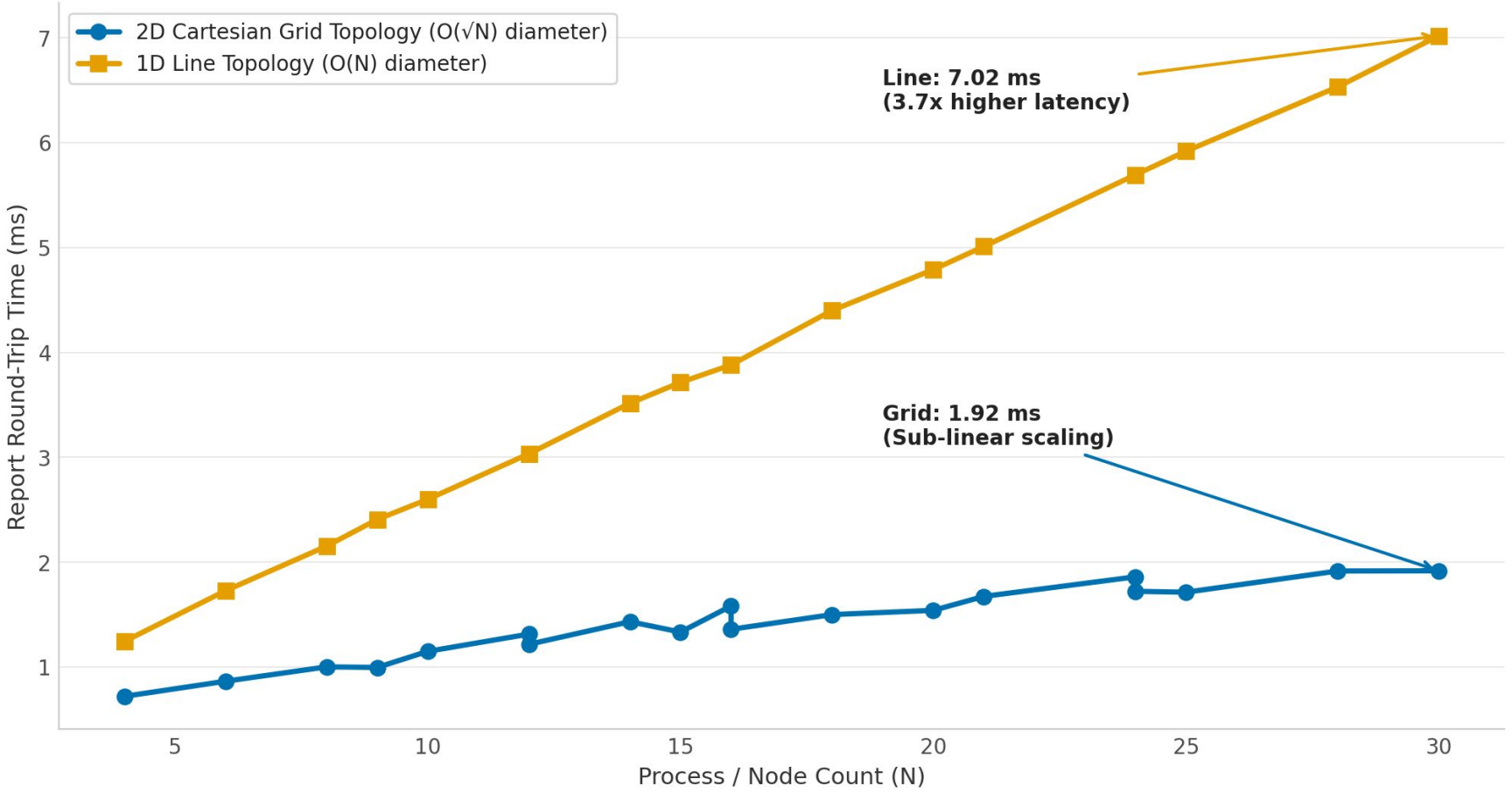
- Executed across 60 distinct topologies (30 2D grids: 2 x 2 → 8 x 8
  30 1D lines: 1 x 2 → 1 x 31)
- 180 total runs on the CAAS cluster environment

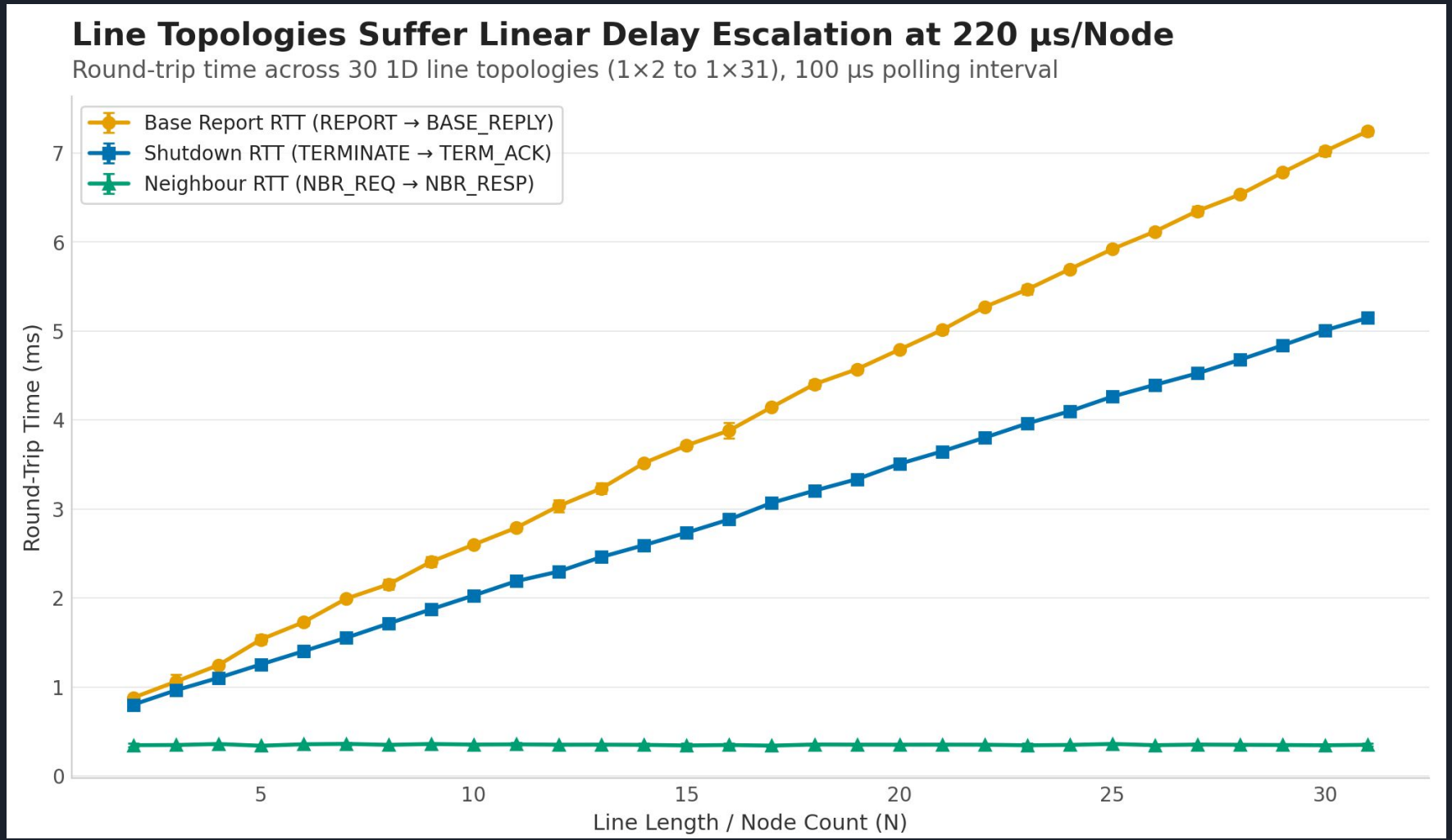
Low Polling Overhead:

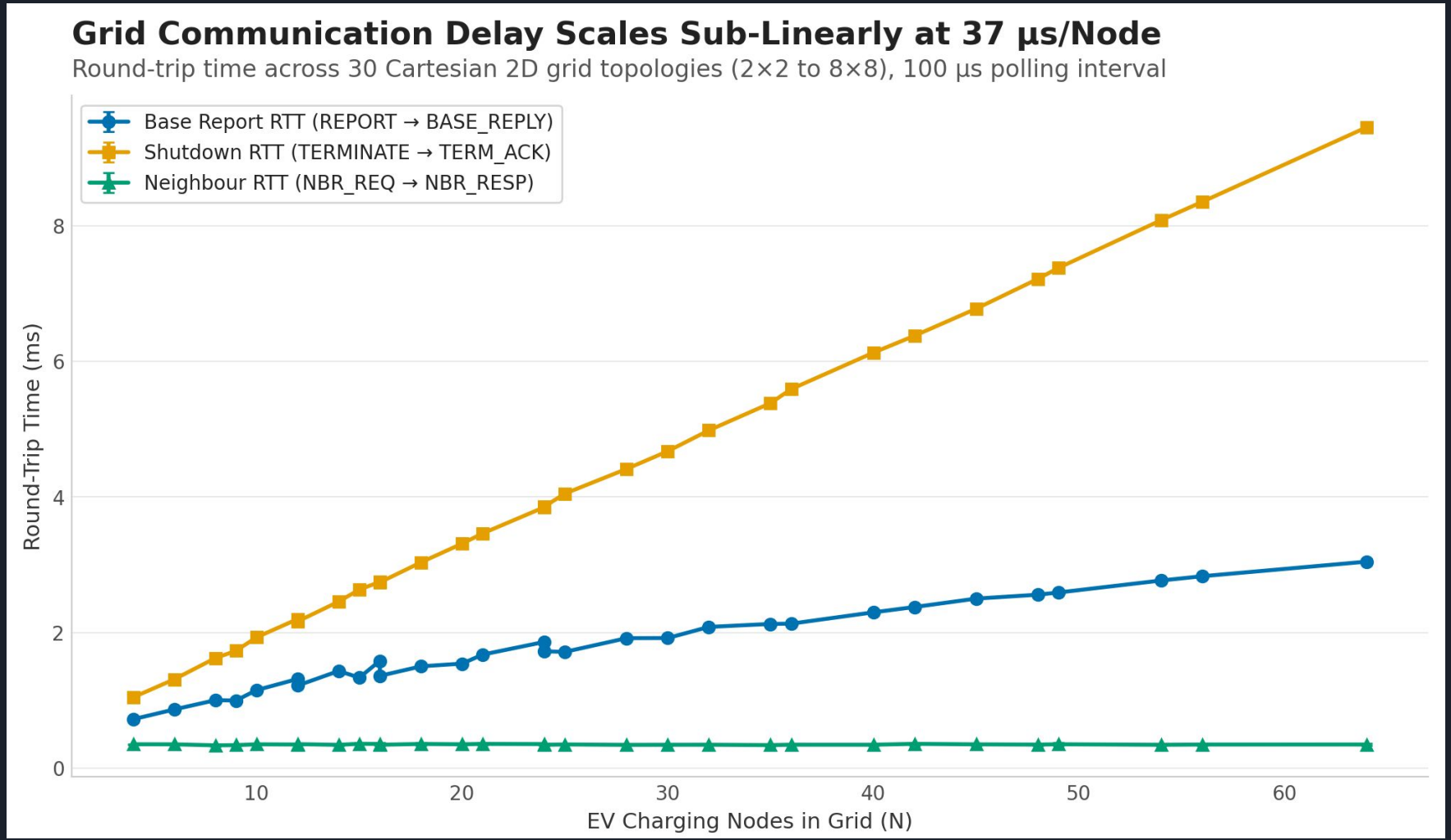
- Configured POLL_US = 100 (100μs) to keep idle wait ≤ 50μs, isolating true network transport delay

2D Grids Outperform Line Topologies by 3.7x at 30 Nodes

Base Station Report RTT comparison for identical process counts (N=4 to 30)




Line Topologies Suffer Linear Delay Escalation at 220 µs/Node
Round-trip time across 30 1D line topologies (1×2 to 1×31), 100 µs polling interval
Base Report RTT (REPORT → BASE_REPLY)
Shutdown RTT (TERMINATE → TERM_ACK)
Neighbour RTT (NBR_REQ → NBR_RESP)
Round-Trip Time (ms)
0
1
2
3
4
5
6
7
5
10
15
20
25
30
Line Length / Node Count (N)


Grid Communication Delay Scales Sub-Linearly at 37 µs/Node
Round-trip time across 30 Cartesian 2D grid topologies (2×2 to 8×8), 100 µs polling interval
Base Report RTT (REPORT → BASE_REPLY)
Shutdown RTT (TERMINATE → TERM_ACK)
Neighbour RTT (NBR_REQ → NBR_RESP)
Round-Trip Time (ms)
0
2
4
6
8
10
20
30
40
50
60
EV Charging Nodes in Grid (N)

Delay Scaling: 2D Grid vs 1D Line (Q3a & Q3b)

Question 3a (2D Grid Scaling):

- Neighbour RTT: $O(1)$ constant at 0.35 ms (0 µs / node)
- Base Report RTT: Sub-linear $O(\sqrt{N})$ scaling at 37 µs / node (0.72 ms at 2 x 2 → 1.92 ms at N = 30)

Question 3b (1D Line Topology Scaling):

- Base Report RTT: Strict linear $O(N)$ scaling at 220 µs / node (5.9 x faster growth than 2D growth due to $O(N)$ network diameter D = N)
- At N = 30 nodes: Line latency reaches 7.02 ms vs 1.92 ms in 2D Grid (3.7 x higher latency)

Executive Summary of Results

Metric	Message Pair	2D Grid Scaling (O(√N) Diameter)	1D Line Scaling (O(N) Diameter)	Key Technical Insight
Neighbour RTT	NBR_REQ → NBR_RESP	0.35ms (*O*(1) flat)	0.35ms (*O*(1) flat)	Identical 1-hop local exchange
Base Report RTT (N = 30)	REPORT → BASE_REPLY	1.92ms (37 µs / node)	7.02ms (220 µs / node)	Line is 3.7x slower at N = 30
Shutdown RTT	TERMINATE → TERM_ACK	Linear *O*(N) (140 µs / node)	Linear *O*(N) (150 µs / node)	Base Station sequential ACK processing

2D Cartesian grid structure restricts network diameter and hop counts, preventing communication bottlenecks as the network scales

Issues, Limitations & Future Work

Hardware Contention:

- Laptop testing inflates latency due to CPU oversubscription
- Cluster execution (CAAS) isolates true physical network scaling

Centralised Coordinator Bottleneck:

- Single Base Station becomes a processing bottleneck for N > 100 → requires multi-region base stations

State Freshness:

- Periodic report snapshots mean rapid EV arrivals can lead to stale candidate node selection

Future Work:

- Persistent non-blocking communication (MPI_Recv_init), dynamic OpenMP thread scheduling, and real-world EV arrival trace integration

QUESTIONS ?